

Onyx Document Processing Demo

This document demonstrates PDF processing with text, images, and tables.

Core Features:

- Agentic RAG with hybrid indexing
- Deep Research for comprehensive reports
- Custom AI Agents with unique knowledge
- Web Search integration (Serper, Brave, etc.)
- Code Execution in sandboxed environments
- Voice Mode with TTS and STT support
- Image Generation from text prompts
- 50+ Enterprise Connectors

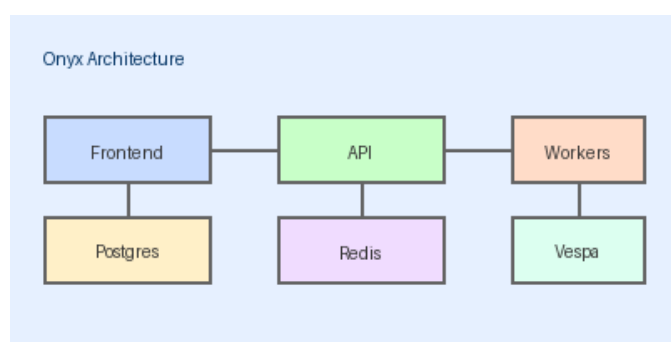


Figure 1: Onyx system architecture diagram

The Onyx platform connects to company documents, apps, and people. It provides RAG-based intelligent Q&A with advanced retrieval.

Technology Stack:

- Backend: Python 3.11, FastAPI, SQLAlchemy, Celery
- Frontend: Next.js, React, TypeScript, Tailwind CSS
- Database: PostgreSQL with Redis caching
- Vector Search: Vespa vector database
- LLM Integration: LiteLLM with multi-provider support

Connector Comparison Table

The following table compares different connector types:

Connector	Type	Real-time	Permissions
Confluence	Cloud	Yes	OAuth + API Token
Google Drive	Cloud	Yes	OAuth 2.0 Service Account
SharePoint	Cloud	Yes	Azure AD OAuth
Slack	Cloud	No	Bot Token
GitHub	Cloud	Yes	Personal Access Token
Jira	Cloud	Yes	API Token + Email
Notion	Cloud	No	Integration Token
Salesforce	Cloud	Yes	OAuth 2.0

Document Processing Pipeline:

Stage 1 - Docfetching: Celery Beat checks connectors every 15s.

Stage 2 - Extraction: Connector fetches documents in batches.

Stage 3 - Processing: Documents are chunked, embedded, and indexed.

Stage 4 - Indexing: Chunks are written to Vespa vector database.

Chunking Strategy:

- Sentence-based chunking with chonkie
- Title prefix + content + metadata suffix
- Mini-chunks for fine-grained retrieval
- Large chunks for context-rich retrieval